

1. b) Data and a pointer to the next node.
2. c) NULL
3. b) Self-referential structure.
4. b)  $head == NULL$
5. b)  $newnode \rightarrow next = head; head = newnode;$
6. b)  $temp = head; head = head \rightarrow next; free(temp);$
7. c) Traversing from tail to head.
8. b)  $n$
9. c) Copying the data of  $P \rightarrow next$  into  $P$ , then deleting  $P \rightarrow next$ .
10. b) while ( $P \neq NULL$ )
11. b)  $P \rightarrow next == NULL$
12. c) Two pointers advancing at different rates.
13. c) Three
14. b) Exactly one node.
15. b)  $P = (struct node *) malloc(sizeof(struct node));$   
Name - Debayan Pandey Roll-036 Sec-A
16. b) Avoid special-case handling for insertion and deletion at the head.
17. c) The first node.
18. b) It does not support random (indexed) access.
19. b) Loss of access to all nodes, causing a memory lock
20. b) A slow pointer and a fast pointer
21. c) the last node.

22. b) Nodes may be scattered anywhere in memory.

23. 

```
- struct node {  
    int data;  
    struct node *next;  
};
```

24. 305070

25. 

```
void insertEnd (struct node **head, int x) {  
    struct node *t = (struct node *) malloc (sizeof (struct  
node));
```

```
t->data = x;
```

```
t->next = NULL;
```

```
struct node *p = *head;
```

```
while (p->next != NULL)
```

```
p = p->next
```

```
p->next = t
```

}

26. 

```
void reverseLL (struct node **head) {
```

```
    struct node *prev = NULL;
```

```
    struct node *curr = *head;
```

```
    struct node *next = NULL;
```

```
    curr = curr->next;
```

```
    curr->next = prev;
```

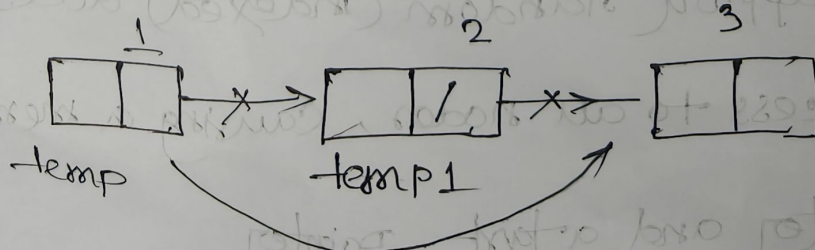
```
    prev = curr;
```

```
    curr = next;
```

```
    return prev;
```

}

27.



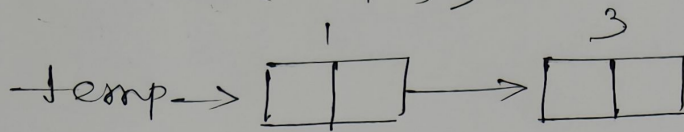


~~temp1 = temp~~ → next

~~temp → next = temp1 → next~~

~~temp1 → next = null~~

free(temp1);



28. i) 7 → 5 → 8 → 12 → 3 → NULL

ii) 7 → 5 → 8 → 3 → NULL

iii) 7 → 5 → 8 → 3 → 9 → NULL

Name - Debayan Pandey Roll - 036 Sec - A